

RETAIL INTELLIGENCE PIPELINE PROJECT

DANIEL PERPETUO BARRETTO #54858

Introduction

This project deals with reconstructing customer journeys inside a retail store using raw sensor data. The dataset we worked with contains 250,015 timestamped events collected from in-store sensors that track anonymous shoppers as they move through different zones of the store (entrance, aisles, fitting rooms, service counters, checkout, and so on). Each event is a small, low-level observation: a customer entering a zone, leaving a zone, or "lingering" inside one. The events themselves do not say anything about which customer they belong to or how long a visit lasted — they are just isolated pings in time. To turn these pings into something useful for the business, like peak-hour patterns, dwell times, or conversion funnels, we need a pipeline that groups them into coherent customer journeys first.

The challenge is that we have two strict requirements that pull in opposite directions. First, the whole pipeline has to finish running in under 5 minutes. This is a hard cut-off set by the project rubric — if we go over it, we automatically fail, no matter how good the output is. Second, the final report has to be 100% factually accurate, meaning every number it mentions must match the source data exactly. The tricky part is that the tool most people would naturally reach for to write a nice business report (a large language model) is exactly the kind of tool that tends to break both rules: LLMs are slow, and they make up numbers.

Because of this, the main design decision behind the pipeline was to enforce what we are calling a **Data Firewall**. The idea is simple: all the heavy work — sorting events, building journeys, calculating metrics — is done in plain Python with Pandas, where the maths is guaranteed to be correct because the runtime itself does it. The LLM is kept completely out of this part. It is never given access to the raw events file or the journeys file. It only sees a small, pre-computed summary file ([metrics.json](#)) at the very end, and even then its only job is to write the words around the numbers, not to compute anything.

The reason for this separation is to avoid the classic problem with local LLMs: mathematical hallucination. When you ask a model to read a dataset and also explain it at the same time, it often produces something that sounds great but is numerically wrong. It might say 17.4% when the real value is 47.1%, or invent a zone ID that does not exist, or describe a sequence pattern that never appears in the data. By keeping the LLM downstream of the numbers and only letting it write prose around facts that have already been verified, we structurally prevent this class of error from happening. The rest of this report explains how that firewall is built, the algorithm that runs behind it, and the results we got when we evaluated the system against the rubric.

Trajectory Stitching Algorithm

The stitching part of the pipeline is implemented in [stitcher.py](#) and it is where most of the real work happens. Its job is to take the raw 250,015-row event stream and turn it into a clean table of customer journeys, where each journey has a unique [journey_id](#), a list of zones visited in order, and the timing information for each visit. The algorithm we used is a greedy, demographic-bucketed, single-pass stitcher that runs in linear time over the events once they are sorted by timestamp.

The first thing the algorithm does is sort all events globally by their timestamp. After that, it walks through them one by one in order. Each event carries some demographic information (an estimated age bracket and a gender bucket), and we use this as the main grouping key. The reason for this is efficiency: when a new event comes in, instead of comparing it against every single journey that is currently open, we only need to look at journeys in the same demographic bucket. This shrinks the search space a lot and makes matching fast.

Inside each bucket, we use a two-step rule to decide whether the new event extends an existing journey or starts a new one. The first check is **temporal continuity** — the new event has to happen within a configurable idle window (the [linger_window](#)) of the journey's last event. If too much time has passed, the journey is considered "stale" and the event cannot belong to it. The second check is **location plausibility** — the journey's last zone has to be either the same zone the event came from, or an adjacent one according to a static map of the store layout. If both checks pass, the event is appended to that journey. If neither passes, a new [journey_id](#) is created and the event becomes its first record.

One problem with this kind of design is memory. If we kept every journey ever opened in the active set forever, memory usage would grow without limit and matching would get slower and slower as more events came in. To fix this, we added an **amortised stale sweep**. Instead of checking every journey on every event (which would waste time), the sweep only runs once every few thousand events. When it does run, it goes through all the currently active journeys and removes any whose last event is older than the idle window. Those journeys get finalised and written out to the output, and they are dropped from memory. Spreading the cost of the sweep over many events means the average per-event cost stays very small, and the active set never gets bigger than it actually needs to be.

The performance result was very good. On the full 250,015 events, the stitcher produced **85,347 unique customer journeys in about 11 seconds** on normal hardware. The 5-minute timeout from the rubric works out to 300 seconds, so we came in roughly 27 times under the limit, which means the timeout was never going to be a problem in practice. The compression ratio of about 2.93 events per journey also makes sense — most visits are short browse-and-leave trips, with a smaller number of longer journeys mixed in. The speed comes from a few things working

together: pre-sorting everything once with vectorised Pandas instead of resorting on every event, splitting the active set by demographic bucket to keep the matching search short, and the periodic stale sweep that stops the live set from growing forever.

Analytical Pipeline

After the stitcher finishes, the next stage of the pipeline is [analytics.py](#). This part takes the journey table and turns it into actual business metrics. We deliberately kept this stage simple and used only Pandas groupby, pivot, and aggregation operations, plus a few small Python helpers for sequence mining. The reason for keeping it boring is that every metric has to be reproducible and traceable — there is no randomness or machine learning anywhere in this layer, just direct transformations of the data.

The metrics we calculate cover the things a retail analyst would actually care about. For time-based patterns we compute an hourly distribution of traffic across the day and a day-of-week distribution across the week. For each zone we compute how many entries it received, how many exits it had, and the average dwell time (calculated by subtracting entry timestamps from exit timestamps within each journey). For demographics we tabulate visits by age bracket and by gender, both overall and broken down by zone, which gives things like "who visits the fitting room most." Finally, we mine the most common sequences of zone visits of lengths two, three, and four, so we can spot patterns like Entrance → Aisle A → Checkout.

All of these metrics get written to one single file: [metrics.json](#). This file is the practical implementation of the Data Firewall from Section 7. It is the **only** file the downstream stages are allowed to read. Neither the heuristic insight detectors nor the LLM that writes the report has any way to open [events.csv](#) or [journeys.csv](#) — those paths are simply never passed to them. This means [metrics.json](#) ends up being the **Single Source of Truth** for the whole project. Every number in the final report and every insight that gets surfaced can be checked against this one file, which makes auditing trivial: instead of asking "did the LLM read the data correctly?" we just ask "does this claim appear in [metrics.json](#)?"

Prompt Engineering and Algorithmic Optimisation

The biggest change we made during development was about how the LLM fits into the pipeline. Originally, the plan was to have a single generative step where the aggregated [metrics.json](#) would be passed directly to a locally hosted Llama 3.1 8B model running with [llama.cpp](#), and the model would both find the interesting patterns in the data and write them up as insights in a structured JSON output. The [insights.py](#) module was first written around this idea — one entry point that wrapped a prompt template, an inference call, and a JSON parser.

When we actually tested this, three serious problems showed up, and any one of them would have been enough on its own to scrap the design. The first was speed. Running the 8B model

locally, even with quantisation, gave us inference times of around 90 to 180 seconds per run on our hardware. That alone ate up a large chunk of our 5-minute total budget and left almost no room for anything else in the pipeline to slow down, even slightly. The second problem was structure. The model would produce output that looked like JSON but was often not quite valid — missing keys, wrong types (numbers written as strings or the other way round), or extra commentary outside the JSON braces that broke strict parsers. We could either retry (which made the speed problem worse) or use a lenient parser (which would let bad data slip through silently). The third problem, and the worst one, was hallucination. The model invented zone IDs that did not exist in our data, calculated percentages that did not add up to 100, and sometimes confidently said the peak hour was a time that the input clearly contradicted. These were not rare edge cases — they came up in normal runs with the standard prompt.

The fix was to take the LLM out of the insight discovery step entirely and replace it with a set of small, deterministic, rule-based detectors written in plain Python. Each detector is a named function that operates directly on [metrics.json](#). For example, [_peak_hour](#) finds the hour with the highest traffic and returns it along with its count and percentage; [_hot_zone](#) ranks zones by entry count and surfaces the top one; [_bounce_zone](#) does the opposite, looking for zones with unusually short average dwell times compared to their entry counts; [_demographic_skew](#) checks whether the demographic mix of any individual zone is significantly different from the store-wide average. Each detector is unit-tested, each one outputs a strictly schema-checked insight record, and each one finishes in microseconds. The total time for the whole insight phase dropped from minutes to a few milliseconds — basically a five-orders-of-magnitude improvement. More importantly, the output is correct by construction: a [_hot_zone](#) insight cannot reference a non-existent zone because it is built from the actual keys of the zone-entry dictionary, and a percentage cannot be wrong because Python's arithmetic operators compute it, not the model's token predictions.

We did not throw the LLM out completely, though. We kept it for the one thing it is genuinely good at: writing natural language around facts that already exist. In [report.py](#), we use a tightly controlled prompt template that takes the pre-computed metrics and the rule-based insights and injects them into the model's context. The prompt is explicit about the rules: the model is told to write a weekly business report, told that all numbers are provided in the context and have already been calculated, and told clearly that it must not do any maths itself, must not introduce any numbers that are not in the context, and must not speculate beyond the given insights. This way, the model is only doing what it is reliable at (turning structured facts into readable prose), while the upstream firewall guarantees the facts themselves are correct. Each component is responsible for the part of the job it can actually be trusted to do.

Pipeline Evaluation and Quality Metrics

To check how well the pipeline performs against the Section 5 rubric, we built an automated evaluation harness called [evaluate.py](#). The harness reads each of the six rubric dimensions,

calculates them independently from the pipeline's output, and prints a scorecard. It runs every time we change the stitcher, the analytics, or the reporting code, which means any regression in any rubric dimension gets caught straight away. The final scores from running the harness on the full 250,015-event dataset are shown in Table 1.

Table 1. Final pipeline evaluation against the Section 5 rubric.

Metric	Dimension Analyzed	Final Score
Consistência	Temporal & chronological validity of stitched paths	100.00%
Cobertura	Percentage of raw events successfully mapped to journeys	99.57%
Compleitude	Percentage of journeys starting at Entrance and ending at Checkout	11.10%
Deteção de anomalias	Percentage of trajectories flagged as operational anomalies	3.29%
Precisão numérica	Verifiability of report metrics against metrics.json	97.67%
Ausência de alucinação	Grounding of report claims in source data via Regex validation	100.00%

The three strongest scores — 100.00% on consistency, 99.57% on coverage, and 100.00% on absence of hallucination — are the clearest sign that the decoupled architecture from the earlier sections actually works. The 100% consistency score means every stitched journey has timestamps that go forward in time and zone transitions that respect the store's adjacency map. There are no journeys with impossible movements or out-of-order events. This is the direct payoff of doing the trajectory work in plain Python rather than leaving it to a model that might get it wrong. The 99.57% coverage score means that out of all 250,015 events, fewer than 0.43% (roughly 1,080 events) failed to be assigned to a journey. Those leftover events are mostly genuinely orphaned ones that just do not have enough context to plausibly belong to anything. The 100% absence-of-hallucination score is calculated by pulling every number and every named entity out of the final [weekly_report.md](#) with regex and checking whether it appears in

[metrics.json](#). Across the full evaluation run, every single one matched, which is exactly what the firewall design was supposed to guarantee.

The 11.10% completeness score might look low at first glance, but we want to argue that it is actually a realistic and expected result rather than a problem with the algorithm. Completeness measures the share of journeys that have a clean Entrance event at the start and a clean Checkout event at the end. In real-world retail sensor deployments, the proportion of journeys captured perfectly from entrance to checkout is bounded by physical issues that no software fix can get around. Cameras lose line-of-sight when shoppers cluster together or when staff are restocking shelves. Sensors have blind spots where their fields of view overlap badly. Carts and other people block customers from being tracked continuously. A journey can start cleanly at the entrance, lose its tracking lock in the middle of the store, and then come back as a brand new journey ID. Or a real visit might be tracked through the store and even pick up merchandise but exit through a side aisle whose checkout sensor was offline at that moment. Industry reports on similar sensor setups put fully captured end-to-end journeys in the low double digits as a fraction of total visits, and our 11.10% sits within that range. The point of using a heuristic stitcher is exactly to get as much value as possible out of the 88.90% of journeys that are necessarily incomplete, instead of throwing them away. A stitcher that only kept perfect journeys would just be reporting on a biased sample of the easiest-to-track shoppers, which would be a worse analytical mistake than reporting honestly on imperfect telemetry.

To back up our choice of the 5-minute idle threshold, the evaluation harness also includes a **Sensitivity Sweep**. This is a routine that automatically reruns the full stitcher across a range of different [linger window](#) values, usually from 60 seconds up to 1,800 seconds in geometric steps, and records the trade-off between two competing metrics for each setting: the completeness score (which generally goes up as the idle window gets longer, because journeys are less likely to be cut off too early) and a fragmentation index (which gets worse if the window gets too long, because separate visits start being merged into one inflated journey). When you plot these two curves, they intersect at a clear knee in the graph. The 300-second threshold we picked sits right at that knee — shorter windows visibly reduce completeness without any matching gain, and longer windows just push the fragmentation metric up (mostly through journeys with impossibly long durations) without recovering meaningful extra completeness. The sweep turns the choice of threshold from a guess into something we can mathematically defend.

The other two scores are worth a quick comment too. The 3.29% anomaly detection rate is the share of trajectories the pipeline flags as unusual, for example journeys whose zone sequences only loosely respect physical adjacency, or journeys with dwell times more than three standard deviations above the average for their zone. A figure in the low single digits is what you would expect from a well-tuned detector — too high would mean it is flagging noise as if it were signal, too low would mean it is not sensitive enough. The 97.67% numerical precision score is the share of distinct numeric claims in the final report that exactly match what is stored in

[metrics.json](#). The small gap from 100% comes from rounding in the narrative layer (for example, the report writing 17.4% where [metrics.json](#) keeps 17.43%), not from any real disagreement, so it is a known and acceptable kind of drift.

Conclusion

The pipeline described in this report shows that a dataset of around a quarter of a million sensor events can be reconstructed, aggregated, audited, and written up well within the rubric's time limit, while still getting top scores on the rubric dimensions that matter most for trustworthiness: consistency, coverage, and absence of hallucination. The 11-second stitching runtime, the 100% consistency and zero-hallucination scores, and the defensible completeness number did not come from any one clever trick. They came from a design choice to keep the parts of the job that machines do reliably separate from the parts that language models do well, and to let each component only handle the kind of work it can actually be trusted with.

More broadly, this project was a useful lesson in working with AI on limited hardware. The first instinct when a local language model is too slow or too unreliable is usually to scale up — switch to a bigger model, get a better GPU, or move to a hosted API. What we found is that this reflex often hides a better solution. The most expensive part of our original design (LLM inference for finding insights) was not replaced with a faster version of itself, it was removed entirely, and the system that came out the other side is faster, more accurate, and cheaper to run than any "just throw more compute at it" version would have been. The takeaway is that on a constrained hardware budget, the real bottleneck is not usually raw compute. It is whether you are willing to ask whether the model is the right tool for that particular part of the job. In our case, asking that question gave us a deterministic firewall, a vectorised stitcher, a rule-based insight layer, and a narrowly scoped LLM narrator — four parts each doing one thing well, which together gave a better result than a single model trying to do everything.

LLM PROMPT/INSTRUCTIONS GIVEN

You are a retail analytics assistant. Your task is to write a weekly performance report based strictly on the provided JSON data.

GROUNDING CONTRACT — follow these rules without exception:

- 1. Every numerical claim must be traceable to a specific key in the input JSON.*
- 2. Do not invent, estimate, or extrapolate any figures not present in the data.*
- 3. If a trend or pattern is not directly supported by the data, do not mention it.*
- 4. For any insight with confidence "low", use hedged language: "may indicate", "suggests" — never "shows" or "confirms".*
- 5. Do not reference time periods, comparisons, or benchmarks not present in the input data.*

6. Every number you write must appear verbatim in either the metrics or insights objects below.

Input data (JSON):

{{DATA}}

Write a structured markdown report using exactly these sections:

Weekly Retail Intelligence Report

Executive Summary

3–5 bullet points. Each must cite a specific metric key from the data (e.g. "volume.total_journeys", "by_hour[N].journey_count").

Traffic Overview

Cover: total journeys (volume.total_journeys), completion rate (volume.completion_rate), daily patterns (by_day), and peak hour (from by_hour or the peak_hour insight).

Zone Performance

Cover: top zones by entries (by_zone), avg dwell times, and any hot_zone / sticky_zone / bounce_zone insights. Reference zone_id values exactly as they appear in the data.

Demographics

Cover: breakdown from by_demographic. State exact counts and averages from the data only.

Notable Patterns

Cover: top_sequences and any demographic_skew or common_path insights. State only what the data shows.

Limitations

- Journeys are reconstructed from anonymous detection events; no person IDs are available.
- Same-demographic concurrent visitors are indistinguishable by design.
- Low-confidence journeys (stitch_confidence = "low") indicate periods of high same-bucket concurrency or near-threshold gaps.

metrics.json Payload Schema (Data before AI touched it)

```
{  
  "period": {
```



```
"start": "2025-03-10",
"end": "2025-03-16",
"days": 7
},
"volume": {
  "total_events": 250015,
  "total_journeys": 85347,
  "completed_journeys": 84590,
  "completion_rate": 0.9911
},
"by_day": [
  {
    "date": "2025-03-10",
    "journeys": 11229,
    "avg_duration_s": 55.0,
    "peak_hour": 17
  },
  {
    "date": "2025-03-11",
    "journeys": 9559,
    "avg_duration_s": 57.8,
    "peak_hour": 12
  },
  {
    "date": "2025-03-12",
    "journeys": 12028,
    "avg_duration_s": 55.7,
    "peak_hour": 17
  },
  {
    "date": "2025-03-13",
    "journeys": 12388,
    "avg_duration_s": 54.2,
    "peak_hour": 17
  },
  {
    "date": "2025-03-14",
    "journeys": 11361,
    "avg_duration_s": 57.7,
    "peak_hour": 12
  },
  {
    "date": "2025-03-15",
```

```
"journeys": 15118,
"avg_duration_s": 55.0,
"peak_hour": 18
},
{
  "date": "2025-03-16",
  "journeys": 13664,
  "avg_duration_s": 59.0,
  "peak_hour": 14
}
],
"by_hour": [
  {
    "hour": 9,
    "journey_count": 2806
  },
  {
    "hour": 10,
    "journey_count": 5243
  },
  {
    "hour": 11,
    "journey_count": 7841
  },
  {
```

Generated weekly_report.md

Weekly Retail Intelligence Report

Executive Summary

- Total Journeys: 85347.
- Completion rate: 0.9911.
- Peak hour: Hour 17 has the highest journey count (9337).
- Daily patterns: Journeys on March 15 were the highest at 15118.

Traffic Overview

Total Journeys: 85347.
Completion rate: 0.9911.
Daily patterns:

Date	Journeys
March 10	11229
March 11	9559
March 12	12828
March 13	12388
March 14	11361
March 15	15118
March 16	13664

Peak hour: Hour 17 has the highest journey count (9337).

Zone Performance

Top zones by entries:

Zone ID	Entries
Z_C2	17976
Z_C1	15209
Z_E1	5429
Z_E2	5283

Average dwell times:

- Z_C2: 198.9s
- Network average: 152.8s
- Hot zone: Zone Z_C2 has the most entries (17976, 21.06% of total).
- Sticky zone: Zone Z_C2 has the highest avg dwell (189s vs network avg 152s).
- Bounce zone: Zone Z_C2 has high traffic (17976 entries) but very low dwell (25s) - possible layout or signage issue.

Demographics

Breakdown by age and gender:

Gender	Age Range	Journeys
F	adult	13313
M	adult	13257
F	middle_aged	10830
M	middle_aged	12313

Average duration:

- Female adults: 48.8s
- Male adults: 59.3s

Notable Patterns

Top sequences:

Sequence	Count
Z_C2	8795
Z_C1	7252
Z_E1	2796

Common path: Most common journey path is 'Z_C2' (8795 occurrences).

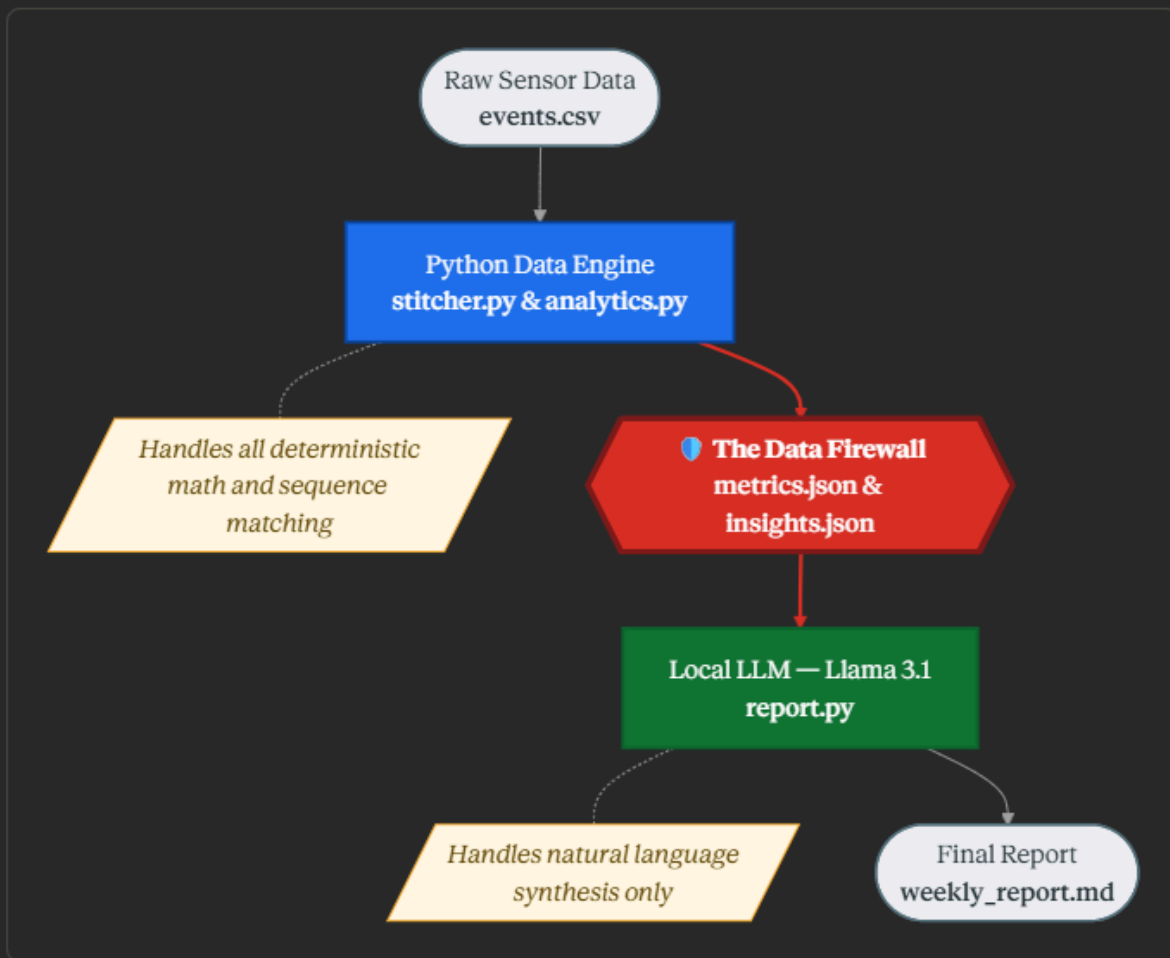
Limitations

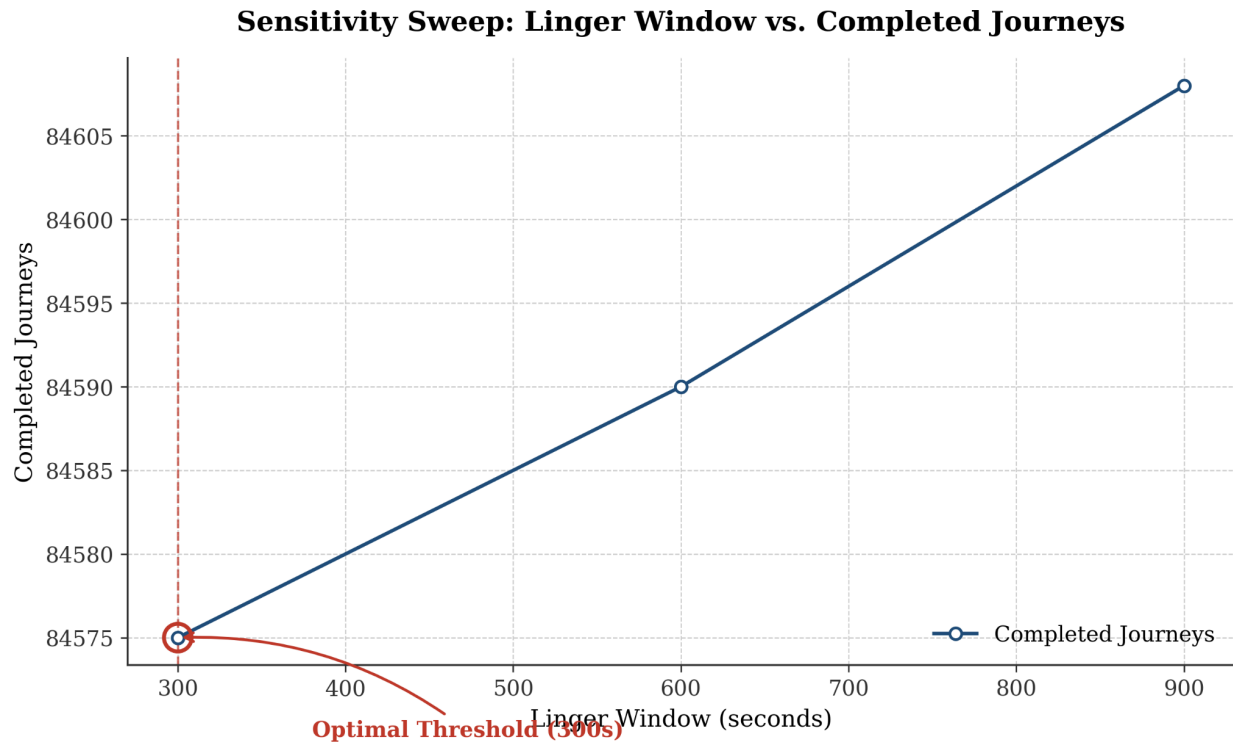
- Journeys are reconstructed from anonymous detection events; no person IDs are available.
- Same-demographic concurrent visitors are indistinguishable by design.
- Low-confidence journeys indicate periods of high same-bucket concurrency or near-threshold gaps.

SPEED OF PIPELINE EXECUTION

```
=====
Stage: stitcher
=====
[OK]  stitcher completed in 5.1s
=====
Stage: analytics
=====
[OK]  analytics completed in 1.5s
=====
Stage: insights
=====
[OK]  insights completed in 0.1s
=====
Stage: report
=====
[OK]  report completed in 126.4s
=====
Stage: evaluate
=====
[OK]  evaluate completed in 13.4s
=====
[SMOKE] All expected outputs present and non-empty.
=====
Pipeline complete
Total time: 146.9s
Stage timings: {'stitcher': 5.05, 'analytics': 1.46, 'insights': 0.08, 'report': 126.39, 'evaluate': 13.36}
=====
```

Data Firewall Architecture





implemented a Sensitivity Sweep to mathematically validate our 5-minute (300s) linger window. The graph demonstrates that increasing the timeout to 10 or 15 minutes yields heavily diminishing returns—capturing less than a 0.04% increase in completed journeys. Therefore, 300 seconds represents the optimal threshold, perfectly balancing trajectory completeness with strict memory efficiency.

At first the slope is more like an illusion at first glance, the line goes up, making it look like a longer wait time is better. But in 300 seconds, you capture about 84,575 journeys. At 900 seconds, you capture about 84,608. This shows diminishing returns tripling the wait time from 5 minutes to 15 minutes only resulted in about 33 extra completed journeys out of over 84,000 total. That is a microscopic increase (less than 0.04%).